AR525: Reinforcement Learning in Robotics

Initial Presentation — Pick & Throw Operations using RL

Group-3

Rishang Yadav, Bhumika Gupta, Aarya Agarwal, Yajesh Chandra

February 19, 2026

Outline

1 Problem Statement

2 Simulator and RL Domain

3 Baseline: MC-PILOT

4 Our Proposed Explorations

5 Literature Survey

6 Evaluation Plan

Problem Statement

What is the Problem? Why is it Interesting?

Problem

- Traditional pick-and-place (PnP) is **workspace-constrained** — targets must lie within the robot's direct reach.
- **Pick-and-Throw (PnT)** extends this: the robot grasps an object and **throws** it to a distant target, exploiting gravity as an extrinsic resource.
- Requires precise **high-speed coordination** between arm motion and gripper release — even small timing errors cause large landing deviations.
- Object dynamics (mass, air drag, friction, inertia) are **hard to characterise analytically**; a learning-based approach is necessary.

Why is it Interesting?

- **Workspace expansion:** throw objects to locations unreachable by direct arm placement.
- **Time efficiency:** faster than moving the arm to every bin (e.g. industrial waste sorting).
- **Data efficiency challenge:** MFRL solutions (e.g. TossingBot) need **hundreds to thousands** of trials — we aim to do far better.
- **Real-world complexity:** stochastic gripper opening delays, communication non-idealities, and object variability make this a compelling RL benchmark.

Simulator and RL Domain

Simulator and RL Domain

Simulator — Gazebo + ROS (following MC-PILOT, Turcato et al., 2025)

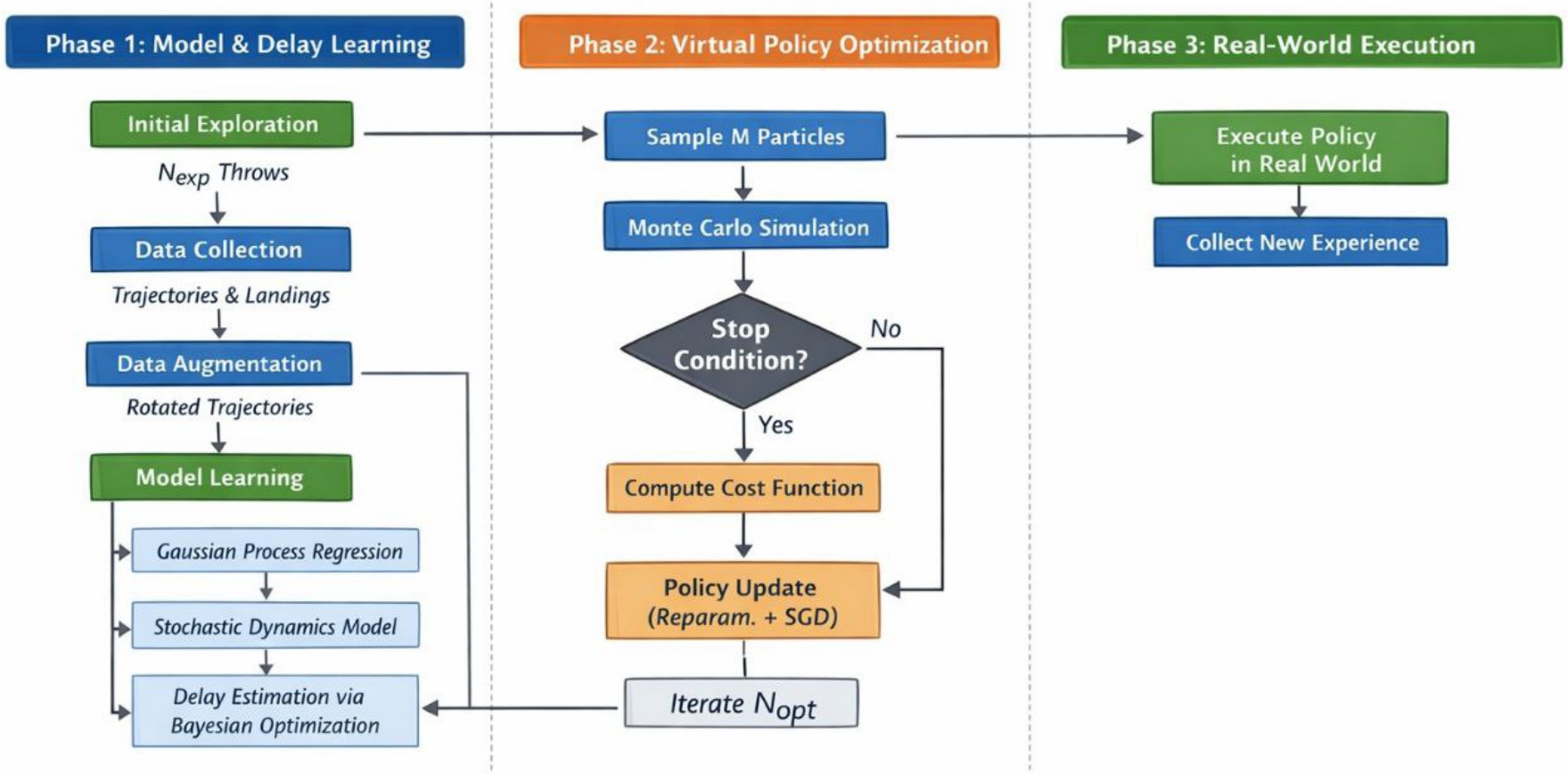
- **Gazebo + ROS**: tight integration provides industrial-grade tools not natively available in alternatives like PyBullet
  - **MoveIt!** — primary robot interface; handles complex motion planning
  - **ros_control** — effort joint trajectory controller for precise velocity coordination at release
- Robot: **6 or 7-DOF robotic arm** (Franka Emika Panda, 7-DOF) for the reach and flexibility needed to throw toward both ground-level and elevated targets
- Objects: rubber ball, tennis ball, cube, cylinder, hammer — varying mass, shape, and inertia

Episode Structure and I/O

- **Input per episode:** [target_position, release_velocity, release_position]
- **Action:** $[z_P, \alpha, v]$ — release height, throw angle and velocity.
- **Output:** [landing_position] — measured via 4-camera RealSense D455 network
- Target domain: cylindrical sector, radius $\ell \in [0.75, 1.75]$ m, angle $\gamma \in [-\pi/6, \pi/6]$; we extend to **variable** z_P above ground

Baseline: MC-PILOT

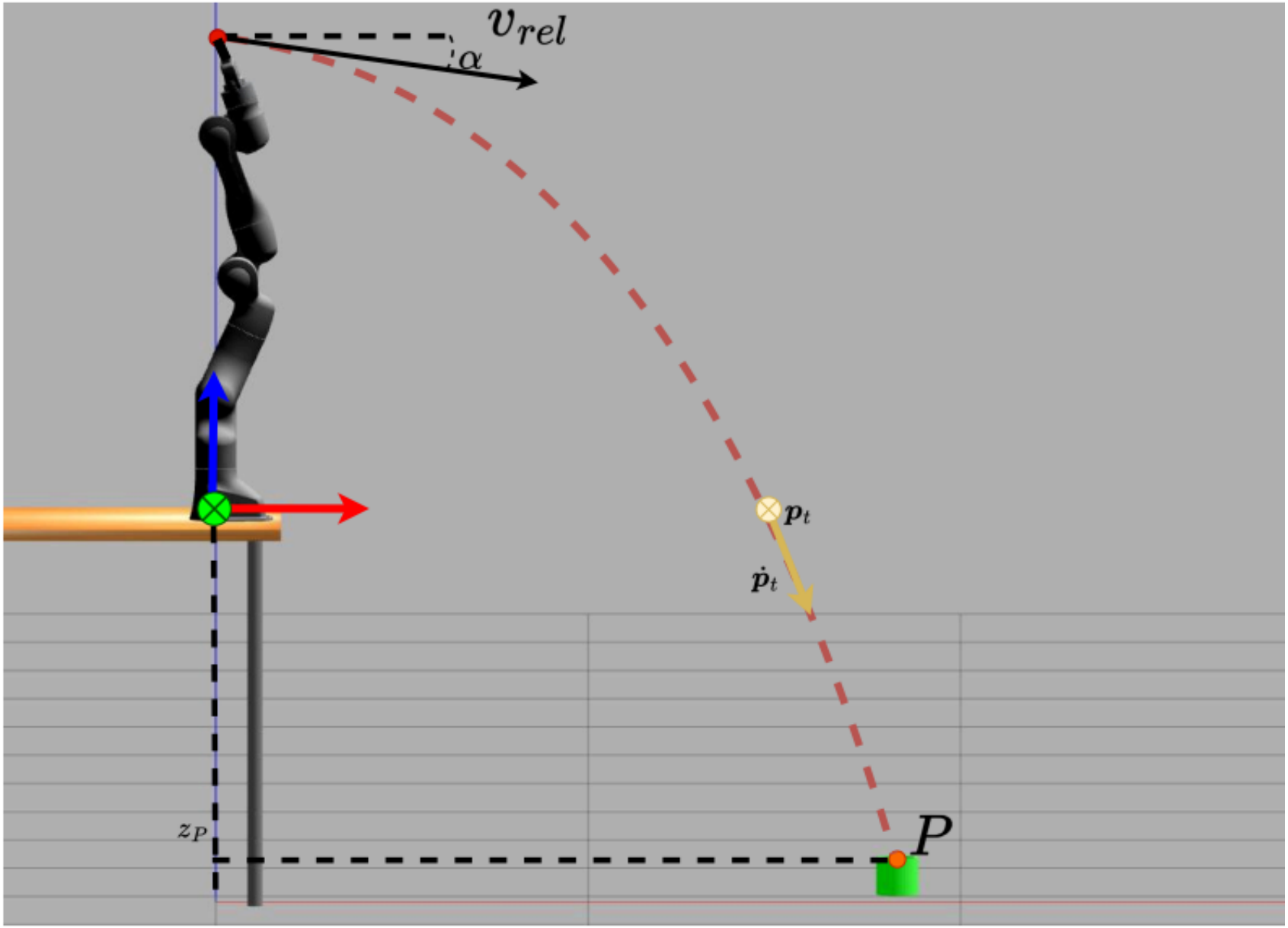
MC-PILOT


Phase 1: Model & Delay Learning
Initial Exploration
N_{exp} Throws
Data Collection
Trajectories & Landings
Data Augmentation
Rotated Trajectories
Model Learning
Gaussian Process Regression
Stochastic Dynamics Model
Delay Estimation via Bayesian Optimization
Phase 2: Virtual Policy Optimization
Sample M Particles
Monte Carlo Simulation
Stop Condition?
No
Yes
Compute Cost Function
Policy Update (Reparam. + SGD)
Iterate N_{opt}
Phase 3: Real-World Execution
Execute Policy in Real World
Collect New Experience

Baseline: MC-PILOT (Turcato et al., 2025)

Core Algorithm

- **Model-Based RL** (MBRL): learns a stochastic **Gaussian Process** model of free-flight object dynamics from $\sim$10 real throws
- Policy $\pi_\theta(P) \to v$: scalar release velocity for a given target P, optimised via **Monte Carlo policy gradient** over GP-simulated particles
- Stochastic **gripper opening delay** $t_d \sim \mathcal{U}(a, a+b)$ estimated via Bayesian Optimisation and propagated through the initial state distribution
- Near-**100% target accuracy** with $\sim$10 exploration throws vs. hundreds for MFRL


v_{rel}
α
p_t
$\dot{p}_t$
z_P
P

Our Proposed Explorations

Fixed Ground-Level Target Plane

Assumption

- MC-PILOT fixes all targets on a **single horizontal ground plane** ($z_P = \text{const}$) with a hard-coded release angle $\alpha \approx 0$ — designed only for downward-trajectory throws.
- When z_P approaches or exceeds $z_{\text{rel}} = 1.50\,\text{m}$, upward momentum is required — fixed q_{rel} and fixed α make this **kinematically infeasible** given Panda's stringent joint limits.

Proposed Modification: Elevated Targets

- **Expanded policy:** $\pi_\theta(P) \to [z_P,\ \alpha,\ v]$ — release height, throw angle, and velocity are **jointly optimised**, allowing dynamic reconfiguration of q_{rel} for each target height.
- **Modified cost:** $\Sigma_c = \text{diag}\left(0,\ \frac{1}{\ell_c},\ \frac{1}{\ell_c}\right)$ minimises lateral (y) and vertical (z) error, extending to **arbitrary target planes**.
- Redefine $\mathcal{D}_P$ to include $z_P > z_{\text{rel}}$; study feasibility boundary within Panda joint limits.

No Wind or Environmental Disturbances

Assumption

- MC-PILOT uses the standard drag equation for **calm, still air** only — all experiments are conducted in a controlled indoor lab with no lateral airflow, gusts, or turbulence.
- In semi-outdoor deployments (warehouses, disaster response), even modest wind can shift a lightweight object's landing beyond the success threshold.

Proposed Modification: Wind Modelling

- Extend Gazebo physics with a **wind force term** added to the drag equation.
- Three wind scenarios: **constant wind**, **random gusts** (step changes in **w**), **turbulence** (Gaussian noise on **w**).
- Quantify accuracy degradation under wind and study whether the GP model can **implicitly absorb** wind effects, or whether an explicit wind-state input is required.

Literature Survey

Literature Survey

Primary Baseline (MBRL for PnT)

- **Turcato et al. (2025)** — *MC-PILOT*: GPR dynamics + Monte Carlo policy gradient + Bayesian delay estimation; near-100% accuracy with $\sim$10 throws
- Amadio et al. (2022) — *MC-PILCO*: parent MBRL algorithm; model-based policy search with Monte Carlo gradient estimation

Model-Free PnT

- Zeng et al. (2020) — *TossingBot*: end-to-end deep RL; effective but requires hundreds of trials

Extended PnT: Cluttered Scenes & Landing Pose

- Kasaei & Kasaei (ICRA 2024) — *Push-Grasp-Throw*: model-free deep RL that combines non-prehensile pushing with grasping and throwing to handle cluttered scenarios; extends the robot's operational reach while isolating target objects via push reconfiguration
- Liu et al. (2025) — *Learning to Throw-Flip*: first work to control landing **pose** (position **and** orientation), not just position; decouples parasitic rotation via the impulse-momentum principle and combines a physics-based free-flight model with regression-based learning for unmodelled effects

Evaluation Plan

Evaluation Plan and Expected Results

Quantitative Metrics

- **Landing Error (m):** mean Euclidean distance between landing point and target.
- **Target Reach Accuracy (%):** fraction of throws within 0.1 m of target.
- **Sample Efficiency:** throws required to reach 80% accuracy — key MBRL vs. MFRL axis.
- **Elevated Target Feasibility:** accuracy vs. target height z_P; identify kinematic boundary where performance degrades.
- **Wind Robustness:** accuracy vs. wind speed $||\mathbf{w}||$ for constant / gust / turbulence; MC-PILOT vs. PPO/SAC degradation curves.

Expected Visualisations

- **Accuracy vs. training throws**: PPO, SAC, MC-PILOT, MC-PILOT + our modifications
- **2D landing scatter plots**: ground vs. elevated targets; calm vs. windy conditions
- **Accuracy vs. wind speed:** degradation onset and relative MBRL/MFRL robustness